

Tema 3. JetPack Compose



Jetpack Compose es un **framework moderno** para crear interfaces gráficas en aplicaciones Android usando **Kotlin**, desarrollado por Google.

Permite diseñar pantallas **sin usar XML**, escribiendo toda la interfaz directamente en código Kotlin.

Menos código y más legible

- ☐ No necesitas escribir XML y luego conectarlo con Kotlin/Java.
- ☐ Todo está en un solo lenguaje (Kotlin) → más simple y directo.

UI declarativa

- ☐ Tú describes cómo quieres que se vea la interfaz y Compose se encarga de renderizarla.

Reutilización y modularidad

- ☐ Creas funciones `@Composable` reutilizables como bloques de Lego.
- ☐ Facilita mantener y escalar proyectos grandes.

Soporte oficial y futuro de Android

- ☐ Google ha dejado claro que Compose es el camino principal para el desarrollo Android.
- ☐ Recibe actualizaciones y mejoras constantes.





Jetpack Compose es un **framework moderno** para crear interfaces gráficas en aplicaciones Android usando **Kotlin**, desarrollado por Google.

Permite diseñar pantallas **sin usar XML**, escribiendo toda la interfaz directamente en código Kotlin.

Menos código y más legible

- ☐ No necesitas escribir XML y luego conectarlo con Kotlin/Java.
- ☐ Todo está en un solo lenguaje (Kotlin) → más simple y directo.

UI declarativa

- ☐ Tú describes cómo quieres que se vea la interfaz y Compose se encarga de renderizarla.

Reutilización y modularidad

- ☐ Creas funciones `@Composable` reutilizables como bloques de Lego.
- ☐ Facilita mantener y escalar proyectos grandes.

Soporte oficial y futuro de Android

- ☐ Google ha dejado claro que Compose es el camino principal para el desarrollo Android.
- ☐ Recibe actualizaciones y mejoras constantes.





XML VS JETPACK COMPOSE

```
buildFeatures{  
    viewBinding true  
}
```

```
<com.google.android.material.button.MaterialButton  
    android:id="@+id/btnSave"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Guardar"/>
```



```
Button(onClick = { }) { this: RowScope  
    Text(text: "Finish")  
}
```

```
private lateinit var binding: ActivityFormBinding  
  
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityFormBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    binding.btnSave.setOnClickListener { uploadData() }
```



@Composable

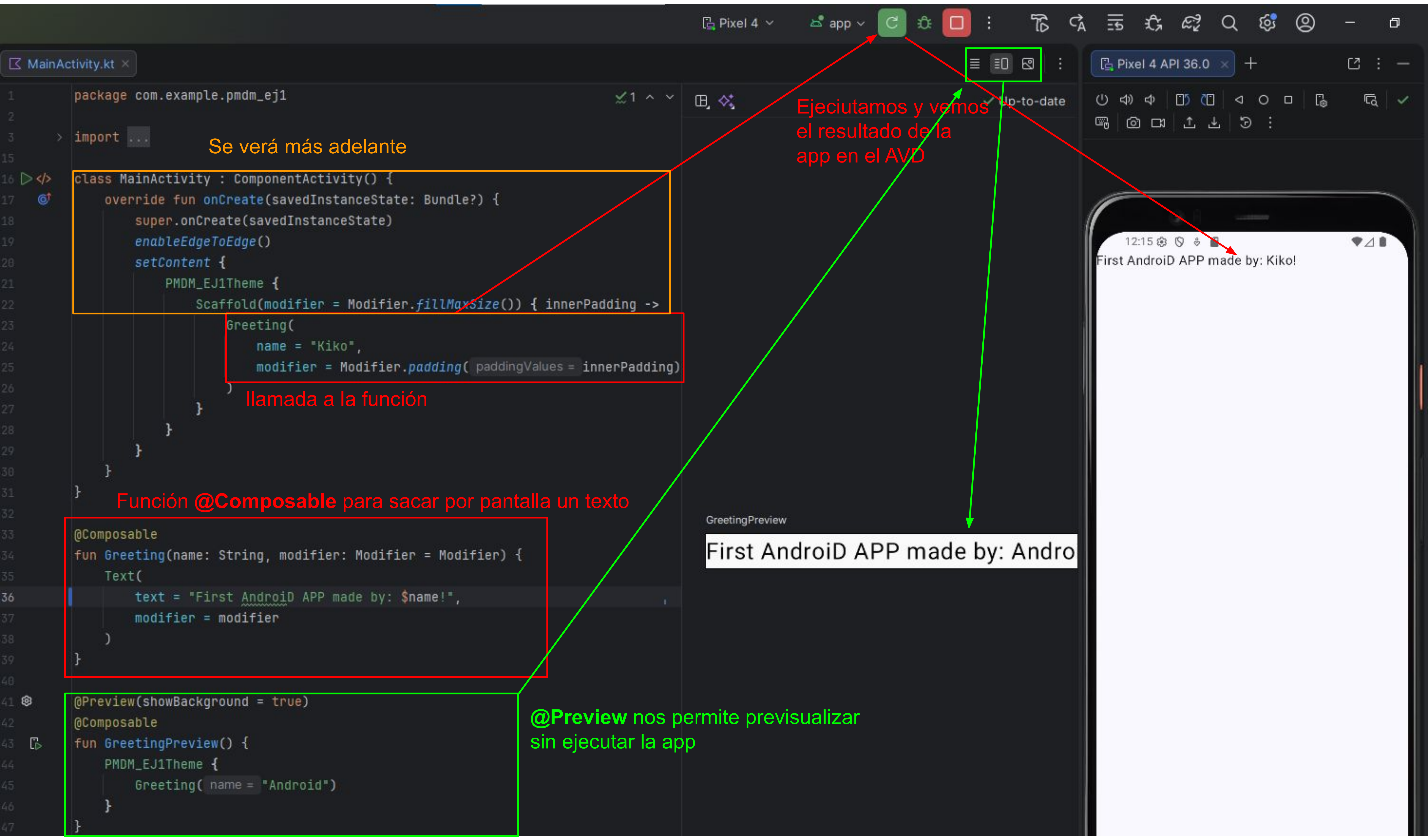
- Es una **anotación en Kotlin** que marca una **función** como parte de la interfaz en **Jetpack Compose**
- Significa que esa función **dibuja algo en pantalla** (texto, botón, layout...).

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

@Preview

- Es una anotación especial que permite ver la UI en Android Studio sin ejecutar la app.
- Solo funciona en funciones @Composable.
- Puedes añadir opciones como mostrar fondo o darle un nombre a la preview.

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    JetPackComposeTheme {
        Greeting(name = "Android")
    }
}
```





Entender la 1ª Aplicación

```
package com.example.pmdm_ej1

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PMDM_EJ1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Kiko",
                        modifier = Modifier.padding(paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}
```

```
class MainActivity : ComponentActivity()
```

- Se define una clase llamada **MainActivity**.: por la **herencia**.
- **ComponentActivity** es la clase base para actividades que usan **Jetpack Compose**.
- **MainActivity** es la Activity en la que entra cuando se abre la app.

```
override fun onCreate(savedInstanceState: Bundle?)
```

- **onCreate()** es el método que se ejecuta cuando la Activity se crea.
- **override** indica que estamos **sobrescribiendo** el método de la clase padre.
- **savedInstanceState** guarda datos si la app se reinicia (por ejemplo, al rotar la pantalla). **Bundle?** significa que puede ser **null** (null safety de Kotlin).

```
super.onCreate(savedInstanceState)
```

- Llama al método original de la clase padre. Es obligatorio para que Android configure correctamente la Activity.

```
enableEdgeToEdge()
```

- Permite que la interfaz se dibuje **debajo de las barras del sistema** (barra de estado y navegación). Se usa en diseños modernos para aprovechar toda la pantalla.



Entender la 1ª Aplicación

```
package com.example.pmdm_ej1

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PMDM_EJ1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Kiko",
                        modifier = Modifier.padding(paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}
```

`setContent {`

- Aquí empieza **Jetpack Compose**.
- `setContent` define **toda la interfaz gráfica** usando funciones `@Composable`.
- Reemplaza el uso antiguo de layouts XML.

`PMDM_EJ1Theme`

- Aplica el **tema visual** de la app (Colores, tipos, estilos...)
- Normalmente se genera automáticamente al crear el proyecto.

`Scaffold(modifier = Modifier.fillMaxSize()) {`
`innerPadding`

- `Scaffold` es un componente estructural de Compose que facilita crear pantallas con:
 - barras superiores e inferiores
 - contenido principal
 - botones flotantes
- `Modifier.fillMaxSize()` hace que ocupe toda la pantalla.
- `innerPadding` evita que el contenido quede tapado por las barras del sistema.



Un **Modifier** se usa para:

- Cambiar **tamaño**
- Añadir **espaciado**
- Cambiar **posición**
- Añadir **eventos** (click, scroll, etc.)
- Aplicar **estilos visuales**

Se pasa normalmente como parámetro llamado **modifier**.

```
@Composable
fun Ejemplo() {
    Text(
        text = "Hola Kotlin",
        modifier = Modifier.padding(16.dp)
    )
}
```

→ añade espacio alrededor del texto

```
Text(
    text = "Hola Compose",
    modifier = Modifier
        .fillMaxWidth()
        .padding(20.dp)
)
```

→ ocupa todo el ancho

→ añade espacio interno



Principales modifiers

Modifier	Función
<code>padding()</code>	Añade espacio
<code>fillMaxWidth()</code>	Ocupa todo el ancho
<code>fillMaxSize()</code>	Ocupa toda la pantalla
<code>size()</code>	Tamaño fijo
<code>background()</code>	Color de fondo
<code>clickable()</code>	Detecta clic

```
@Composable
fun Greeting(
    name: String,
    modifier: Modifier = Modifier
) {
    Text(
        text = "Hola $name",
        modifier = modifier
    )
}
```

Al pasarlo como parámetro, cualquiera que llame a la función puede usar un modifier.

```
Greeting("Ana")
```

Si no queremos usar modifier

```
Greeting(
    "Ana",
    modifier = Modifier.padding(16.dp)
)
```

Usando modifier



Componentes de Jetpack Compose.

App bars

- BottomAppBar
- Bottom Navigation
- AppBar **Nuevo**

Badges

- Badge
- BadgedBox **Nuevo**

Buttons

- Button
- FilledTonalButton
- OutlinedButton
- ElevatedButton
- TextButton
- FloatingActionButton
- SmallFloatingActionButton
- LargeFloatingActionButton
- ExtendedFloatingActionButton
- RadioButton
- IconButton

Cards

- OutlinedCard
- Card
- ElevatedCard

Carousel

- HorizontalPager
- HorizontalMultiBrowseCarousel
- HorizontalUncontainedCarousel

Checkbox

- Checkbox **Nuevo**

Chips

- AssistChip
- FilterChip
- InputChip
- SuggestionChip

Pickers

- Date picker
- Time Picker

Dialogs

- Alert dialog

Divider

- Divider

Lists

- LazyColumn
- LazyRow
- List item

Menú

- DropdownMenuItem
- DropdownMenu

Progress indicator

- CircularProgressIndicator
- Linear Progress Indicator

Sheets

- Bottom Sheet **Nuevo**
- Side Sheet **Nuevo**

Sliders

- Range Slider
- Slider

Snackbar

- Snackbar

Snackbar

- Snackbar

Swipes

- SwipeRefresh **Nuevo**
- SwipeToDismiss **Nuevo**

Switch

- Switch **Nuevo**

Tabs

- Tab
- TabRow

Text

- Text

Text fields

- OutlinedTextField
- TextField

ToolTips

- TooltipBox



Importante!!. No se van a ver todos y cada uno de los componentes, se verán los más básicos y más utilizados, y los que se vayan necesitando en cada proyecto ya se buscarán.

Listado completo:

<https://www.jetpackcompose.pro/home/guide/>



TEXT: Componente más básico, permite representar información.

Hello World

```
@Composable
fun Text(
    text: String,
    modifier: Modifier = Modifier,
    color: Color = Color.Unspecified,
    fontSize: TextUnit = TextUnit.Unspecified,
    fontStyle: FontStyle? = null,
    fontWeight: FontWeight? = null,
    fontFamily: FontFamily? = null,
    letterSpacing: TextUnit = TextUnit.Unspecified,
    textDecoration: TextDecoration? = null,
    textAlign: TextAlign? = null,
    lineHeight: TextUnit = TextUnit.Unspecified,
    overflow: TextOverflow = TextOverflow.Clip,
    softWrap: Boolean = true,
    maxLines: Int = Int.MAX_VALUE,
    minLines: Int = 1,
    onTextLayout: (TextLayoutResult) -> Unit = {},
    style: TextStyle = LocalTextStyle.current
)
```

```
@Composable
fun SimpleText() {
    Text("Hello World")
}
```

Hello World

```
@Composable
fun SimpleText() {
    text = "Hello World ",
    color = Color.Red,
    fontSize = 32.sp,
    fontStyle = FontStyle.Italic,
    fontWeight = FontWeight.Bold
}
```



Ejercicio: Crea un **Composable** llamado **TextoEstilizado()** que muestre un texto con las siguientes características:

- Mostrar el texto **"Aprendiendo Jetpack Compose en clase"**
- Tener **color azul**
- Tamaño de letra **22sp**
- Estar en **negrita**
- Estar en **cursiva**
- Tener **espacio entre letras de 2sp**
- Estar **subrayado**
- Tener **alineación centrada**
Tener **altura de línea de 28sp**
- Estar dentro de un **Modifier** con:
 - **padding(16.dp)**
 - **fillMaxWidth()**
Mostrar **máximo 2 líneas**
- Si el texto se desborda debe mostrar ...

Una vez realizado, estudiar las diferencias entre:

`overflow = TextOverflow.Clip`

`overflow = TextOverflow.Visible`

Modificar para que:

El texto tenga **fondo amarillo**

Use **3 líneas máximo**

Tenga **texto tachado**



TEXTFIELD: Permiten al usuario introducir texto

```
@Composable
fun TextField(
    value: String,
    onChange: (String) -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    readOnly: Boolean = false,
    textStyle: TextStyle = LocalTextStyle.current,
    label: (@Composable () -> Unit)? = null,
    placeholder: (@Composable () -> Unit)? = null,
    leadingIcon: (@Composable () -> Unit)? = null,
    trailingIcon: (@Composable () -> Unit)? = null,
    isError: Boolean = false,
    visualTransformation: VisualTransformation = VisualTransformation.None,
    keyboardOptions: KeyboardOptions = KeyboardOptions.Default,
    keyboardActions: KeyboardActions = KeyboardActions(),
    singleLine: Boolean = false,
    maxLines: Int = if (singleLine) 1 else Int.MAX_VALUE,
    minLines: Int = 1,
    interactionSource: MutableInteractionSource? = null,
    shape: Shape = TextFieldDefaults.TextFieldShape,
    colors: TextFieldColors = TextFieldDefaults.textFieldColors()
): Unit
```

rememberSaveable Guarda el estado aunque se recree la Activity (rotación...)

mutableStateOf() redibuja automáticamente cuando cambia el estado, es necesario para que se vea lo que va poniendo el usuario

```
var text by rememberSaveable { mutableStateOf("") }
```

Permite usar directamente la variable en lugar de text.value

```
TextField(
    value = text,
    onChange = { text = it },
    label = { Text("Label") },
    singleLine = true
)
```

Función Lambda que se ejecuta cuando el usuario introduce texto, en este caso hacemos que la variable text coja el valor que ha puesto el usuario

```
TextField(
    value = text,
    onChange = { text = it },
    label = { Text("Email") },
    placeholder = { Text("example@gmail.com") }
)
```

JETPACK COMPOSE



```
var text by rememberSaveable { mutableStateOf("") }
```

```
TextField(  
    value = text,  
    onChange = { text = it },  
    label = { Text("Label") },  
    leadingIcon = { Icon(Icons.Filled.Favorite, contentDescription = "Localized description") },  
    trailingIcon = { Icon(Icons.Filled.Info, contentDescription = "Localized description") }  
)
```

icono al inicio icono al final

Texto para
accesibilidad (útil en
lectores de pantalla)

JETPACK COMPOSE

```
var password by rememberSaveable { mutableStateOf("") }
var passwordHidden by rememberSaveable { mutableStateOf(true) }

TextField(
    value = password,
    onValueChange = { password = it },
    singleLine = true,
    label = { Text("Enter password") },
    visualTransformation =
        if (passwordHidden) PasswordVisualTransformation() else VisualTransformation.None,
    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
    trailingIcon = {
        IconButton(onClick = { passwordHidden = !passwordHidden }) {
            val visibilityIcon =
                if (passwordHidden) Icons.Filled.Visibility else Icons.Filled.VisibilityOff
            // Please provide localized description for accessibility services
            val description = if (passwordHidden) "Show password" else "Hide password"
            Icon(imageVector = visibilityIcon, contentDescription = description)
        }
    }
)
```

Enter password



Enter password

.....





Ejercicio: Crea un **TextField** con un icono que sea una imagen elegida por ti.

Tips:

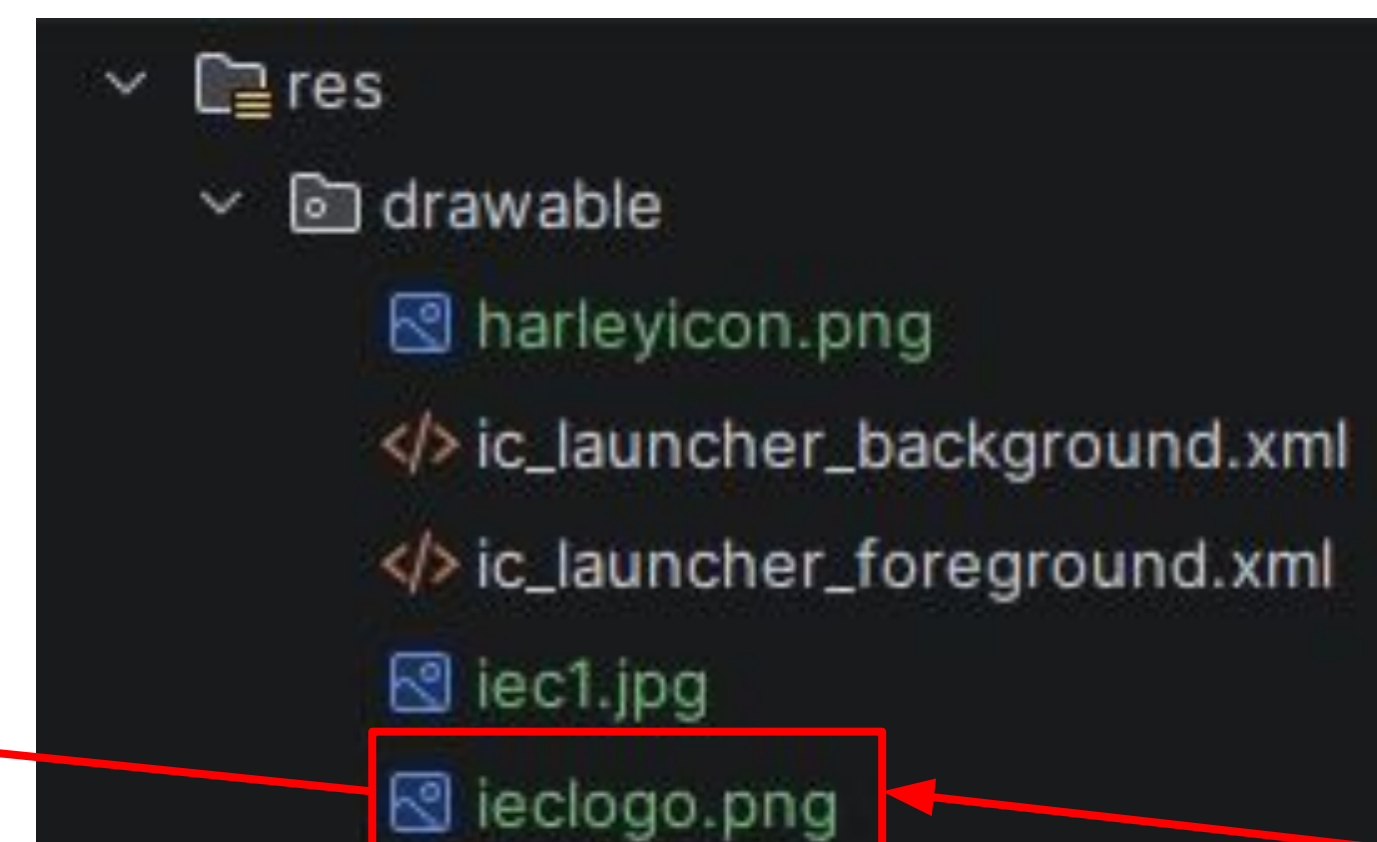
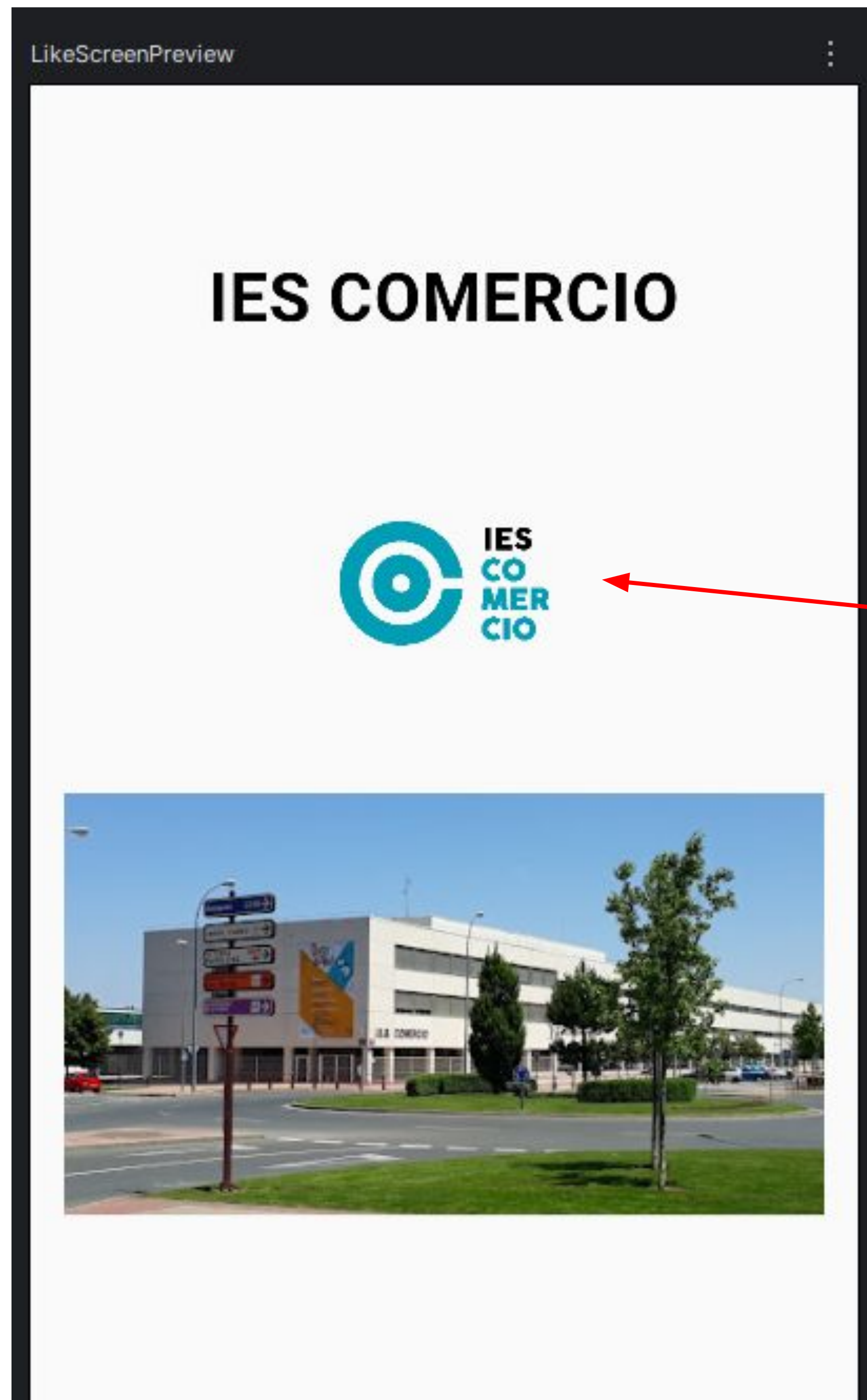
- Coloca la imagen en la carpeta `res/drawable/` del proyecto Android.
- usa el mismo ejemplo que hemos visto pero en lugar de `Icon`, usa la funcion **Image**:

```
Image(  
    painter = painterResource(id = R.drawable.mi_icono),  
    contentDescription = "Mi icono personalizado",  
    modifier = Modifier.size(24.dp) // tamaño del icono  
)
```

- Tendrás que importar Librerías

JETPACK COMPOSE

IMAGE: Permiten al usuario visualizar una imagen



```
Column(  
    modifier = modifier.fillMaxSize().padding(all = 16.dp),  
    verticalArrangement = Arrangement.Top,  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
    Spacer(modifier = Modifier.height(50.dp))  
  
    // Texto IES COMERCIO  
    Text(  
        text = "IES COMERCIO",  
        fontSize = 32.sp,  
        fontWeight = FontWeight.Bold,  
        color = Color.Black,  
        modifier = Modifier.padding(vertical = 16.dp)  
    )  
  
    Spacer(modifier = Modifier.height(50.dp))  
  
    // Imagen pequeña: ieclogo.png  
    Image(  
        painter = painterResource(id = R.drawable.ieclogo),  
        contentDescription = "Logo IES Comercio",  
        modifier = Modifier.size(100.dp)  
    )  
  
    Spacer(modifier = Modifier.height(50.dp))  
  
    // Imagen grande: iec1.jpg  
    Image(  
        painter = painterResource(id = R.drawable.iec1),  
        contentDescription = "Imagen IES Comercio",  
        modifier = Modifier  
            .fillMaxWidth()  
            .height(200.dp),  
        contentScale = ContentScale.Crop  
    )  
}
```

Contenedores

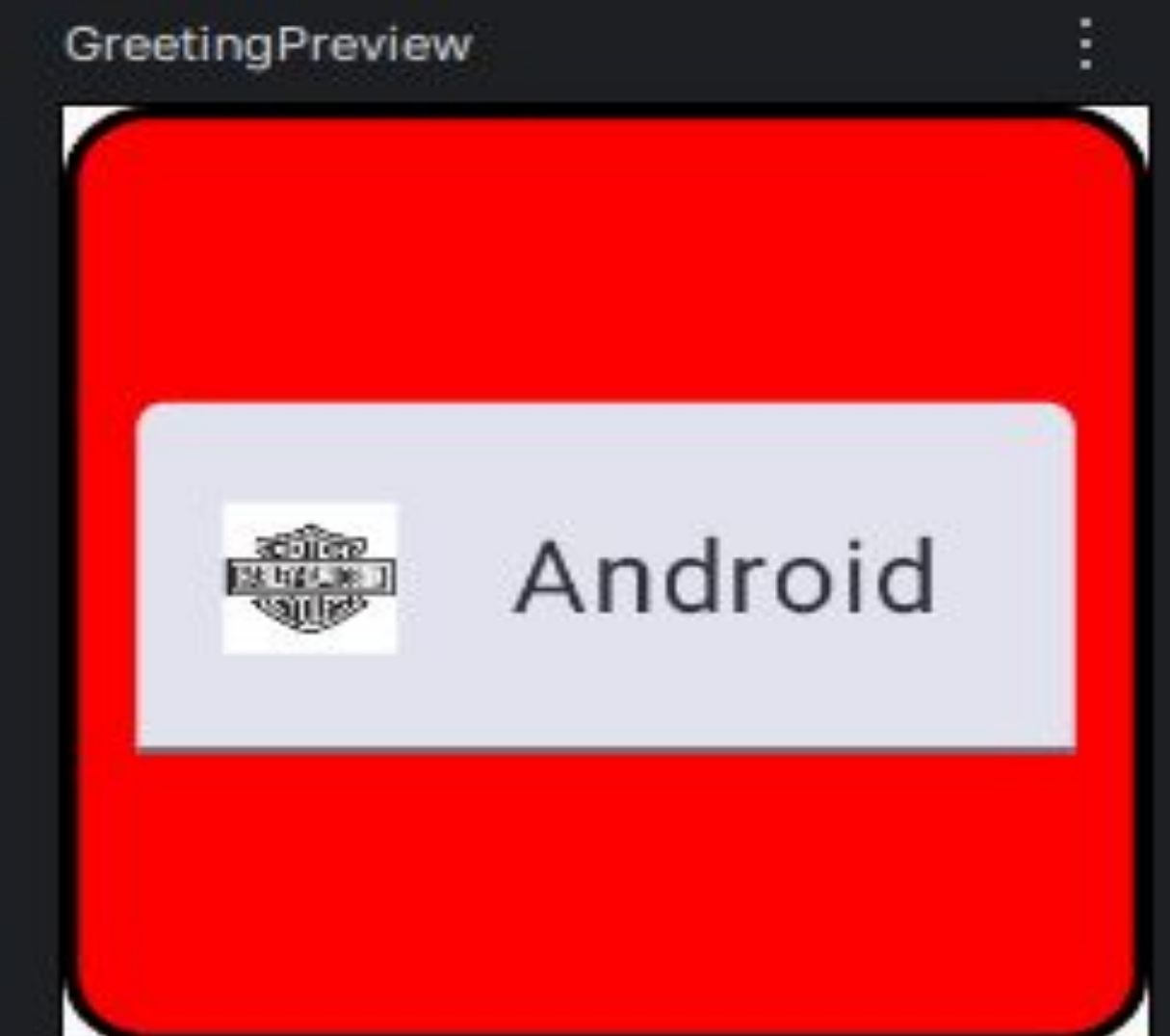
BOX

- En Jetpack Compose, el contenedor Box es muy usado para superponer elementos (similar a FrameLayout en vistas XML) y controlar su alineación interna.
- Su comportamiento se ajusta principalmente a través del parámetro modifier.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {

    Box(
        modifier=modifier
        .size(150.dp)
        .background(Color.Red, shape = RoundedCornerShape(size = 12.dp))
        .border(width = 2.dp, Color.Black, shape = RoundedCornerShape(size = 12.dp))
        .padding(all = 10.dp)
        .clickable{}
        contentAlignment = Alignment.Center
    ) {

        TextField(
            value = name,
            onChange = { },
            //label = { Text("Label") },
            leadingIcon = {
                Image(
                    painter = painterResource(id = R.drawable.harleyicon)
                    contentDescription = "Mi icono personalizado",
                    modifier = Modifier.size(24.dp) // tamaño del icono
                )
            }
        )
    }
}
```



NOTA: Fijarse como al pinchar sobre un elemento en el código se marca en la previsualización y viceversa

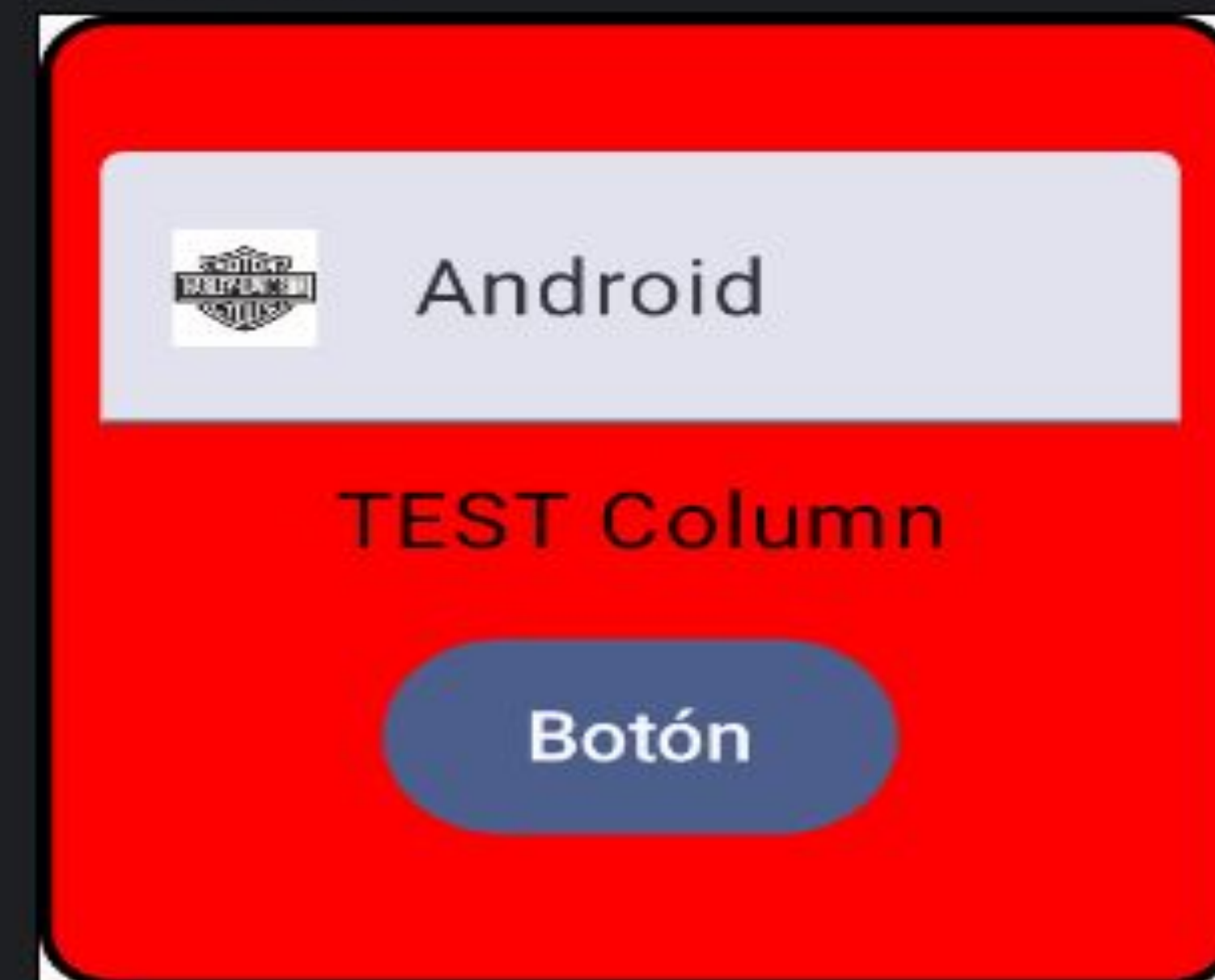
COLUMN

- Es un contenedor que organiza a sus hijos uno debajo del otro (orientación vertical).
- Cumple el mismo papel que un `LinearLayout` en vertical en Android clásico.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {

    Box(
        modifier=modifier
        .size(200.dp)
        .background(Color.Red, shape = RoundedCornerShape(size = 12.dp))
        .border(width = 2.dp, Color.Black, shape = RoundedCornerShape(size = 12.dp))
        .padding(all = 10.dp)
        .clickable{},
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            TextField(
                value = name,
                onChange = { },
                leadingIcon = {
                    Image(
                        painter = painterResource(id = R.drawable.harleyicon)
                        contentDescription = "Mi icono personalizado",
                        modifier = Modifier.size(24.dp)
                    )
                }
            )
            Text(
                text = "TEST Column",
                modifier = Modifier.padding(top = 8.dp)
            )
            Button(
                onClick = { },
                modifier = Modifier.padding(top = 8.dp)
            ) {
                Text("Botón")
            }
        }
    }
}
```

GreetingPreview



JETPACK COMPOSE

ROW

- Coloca sus hijos uno al lado del otro (de izquierda a derecha por defecto).
- Equivale a `LinearLayout` horizontal en XML.

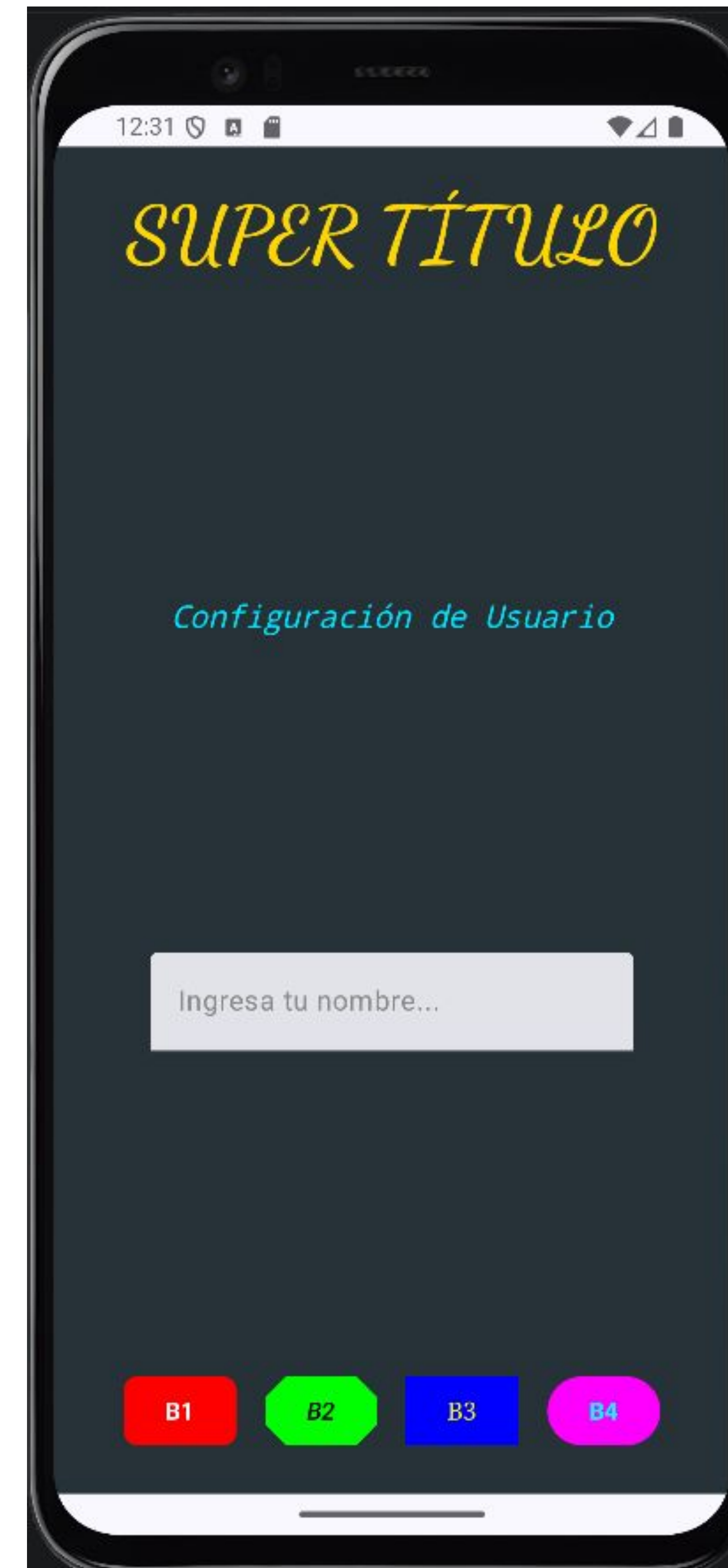
```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .background(Color.LightGray)
            .padding(all = 16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = "item1 ")
        Text(text = "item2. ")
        Button(onClick = { }) {
            Text("OK")
        }
    }
}
```

GreetingPreview

item1 item2. OK

JETPACK COMPOSE

- **Ejercicio:** realiza las siguientes Activitys:





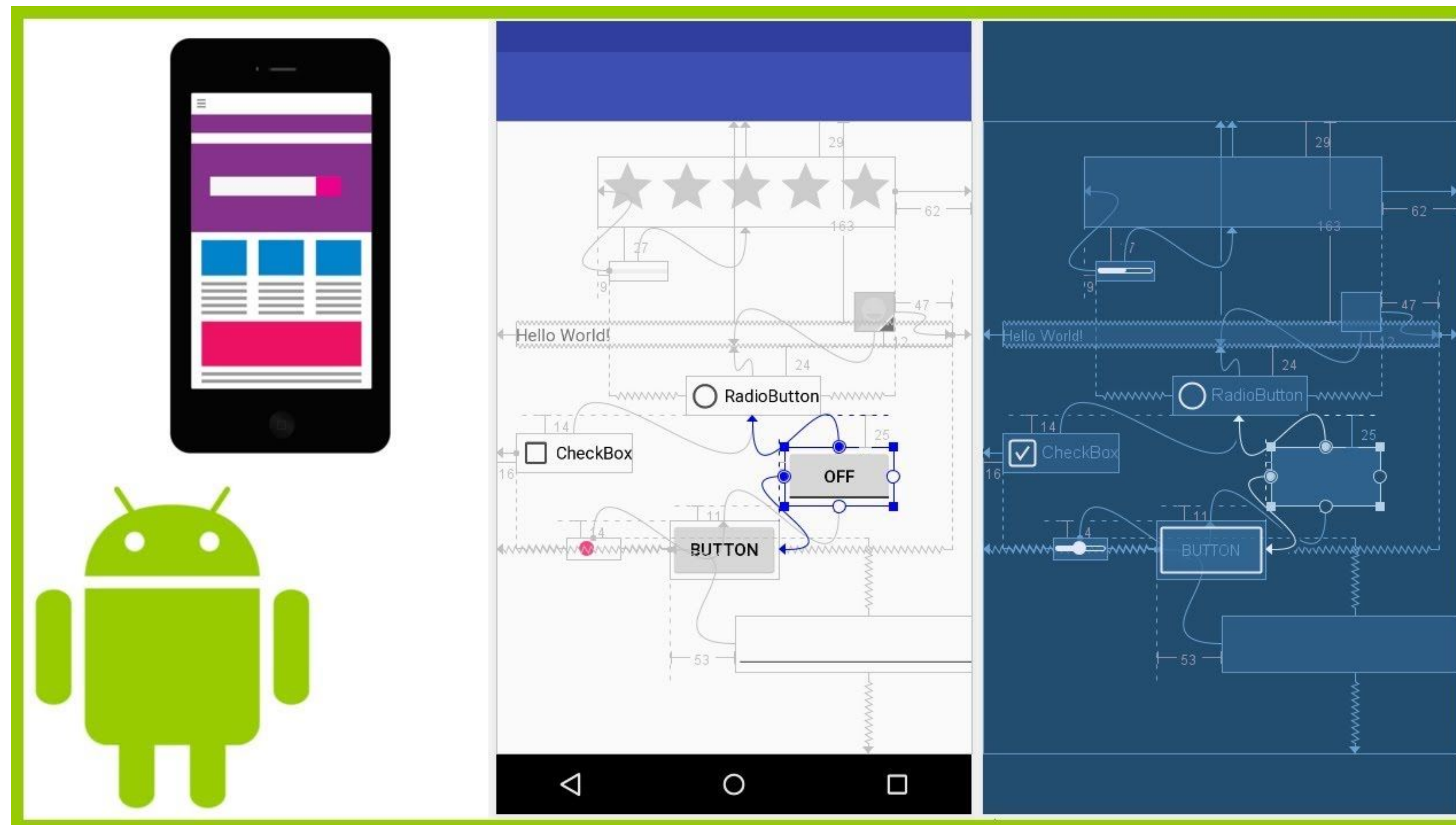
CONSTRAINT LAYOUT EN JETPACK COMPOSE

- Contenedor flexible que permite **posicionar elementos unos respecto a otros** o respecto al contenedor padre.
- Muy útil cuando necesitas diseños más complejos que un Row o Column.

`ConstraintLayout { }` → el contenedor.

`createRefs()` → crea referencias para los hijos.

`constrainAs(ref) { ... }` → establece las restricciones (constraints) de cada hijo.





Constraints más comunes:

`start.linkTo(parent.start)`

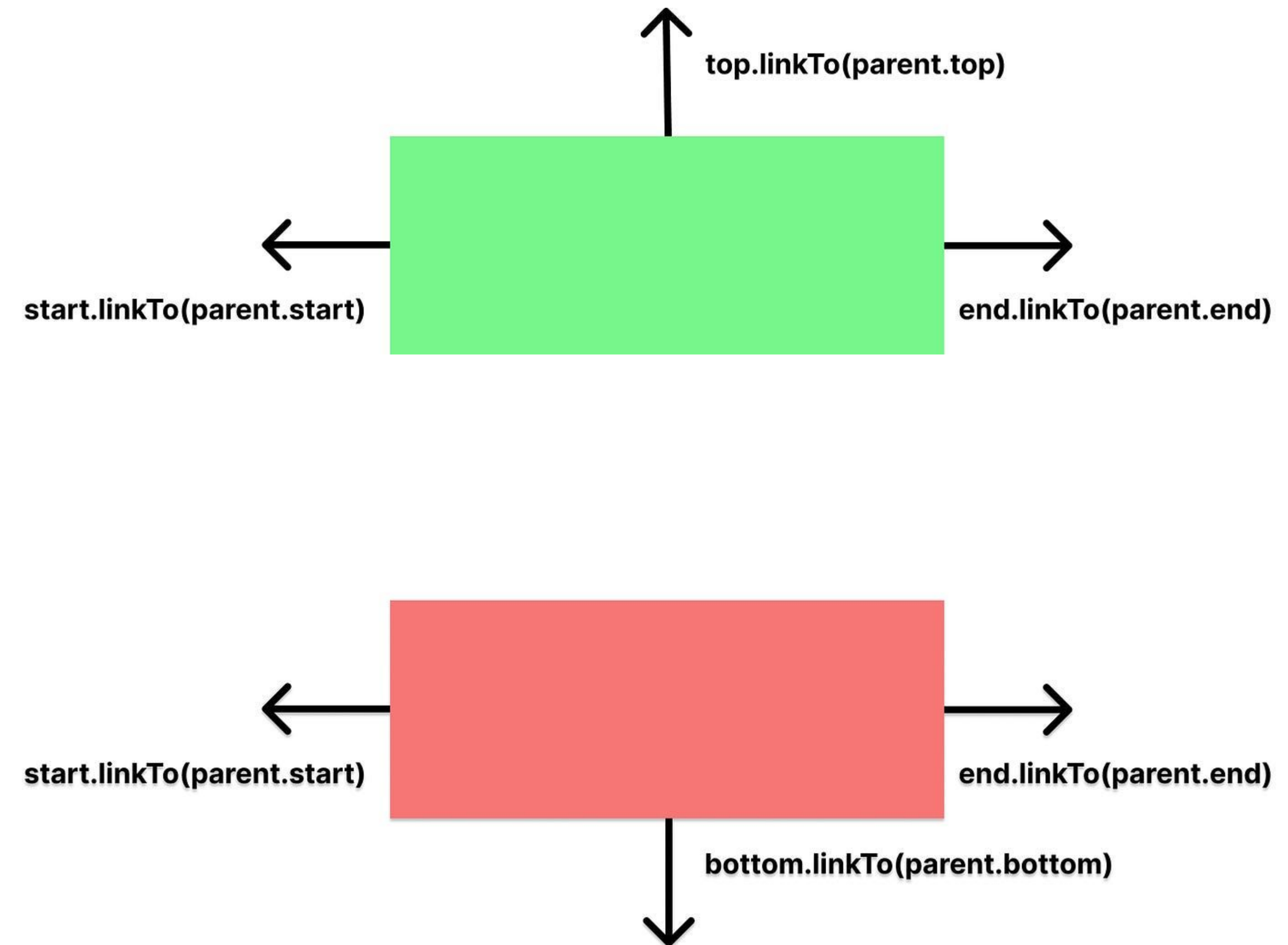
`end.linkTo(parent.end)`

`top.linkTo(parent.top)`

`bottom.linkTo(parent.bottom)`

`centerHorizontallyTo(parent)`

`centerVerticallyTo(parent)`



JETPACK COMPOSE

Añadir dependencias
en el gradle para
layout Constraint

Gradle Scripts

- build.gradle.kts (Pr
- build.gradle.kts (M
- proguard-rules.pro
- gradle.properties (
- gradle-wrapper.pro
- libs.versions.toml (
- local.properties (Sl
- settings.gradle.kts

```
1 [versions]
2 agp = "9.0.1"
3 coreKtx = "1.10.1"
4 junit = "4.13.2"
5 junitVersion = "1.1.5"
6 espressoCore = "3.5.1"
7 lifecycleRuntimeKtx = "2.6.1"
8 activityCompose = "1.8.0"
9 kotlin = "2.0.21"
10 composeBom = "2024.09.00"
11 constraintlayoutCompose = "1.1.1"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
26 androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
27 androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
28 androidx-constraintlayout-compose = { group = "androidx.constraintlayout", name = "constraintlayout-compose", version.ref = "constraintlayoutCompose" }
29
30 [plugins]
31 android-application = { id = "com.android.application", version.ref = "agp" }
32 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

```
androidx-constraintlayout-compose = { group = "androidx.constraintlayout", name = "constraintlayout-compose", version.ref = "constraintlayoutCompose" }
```

JETPACK



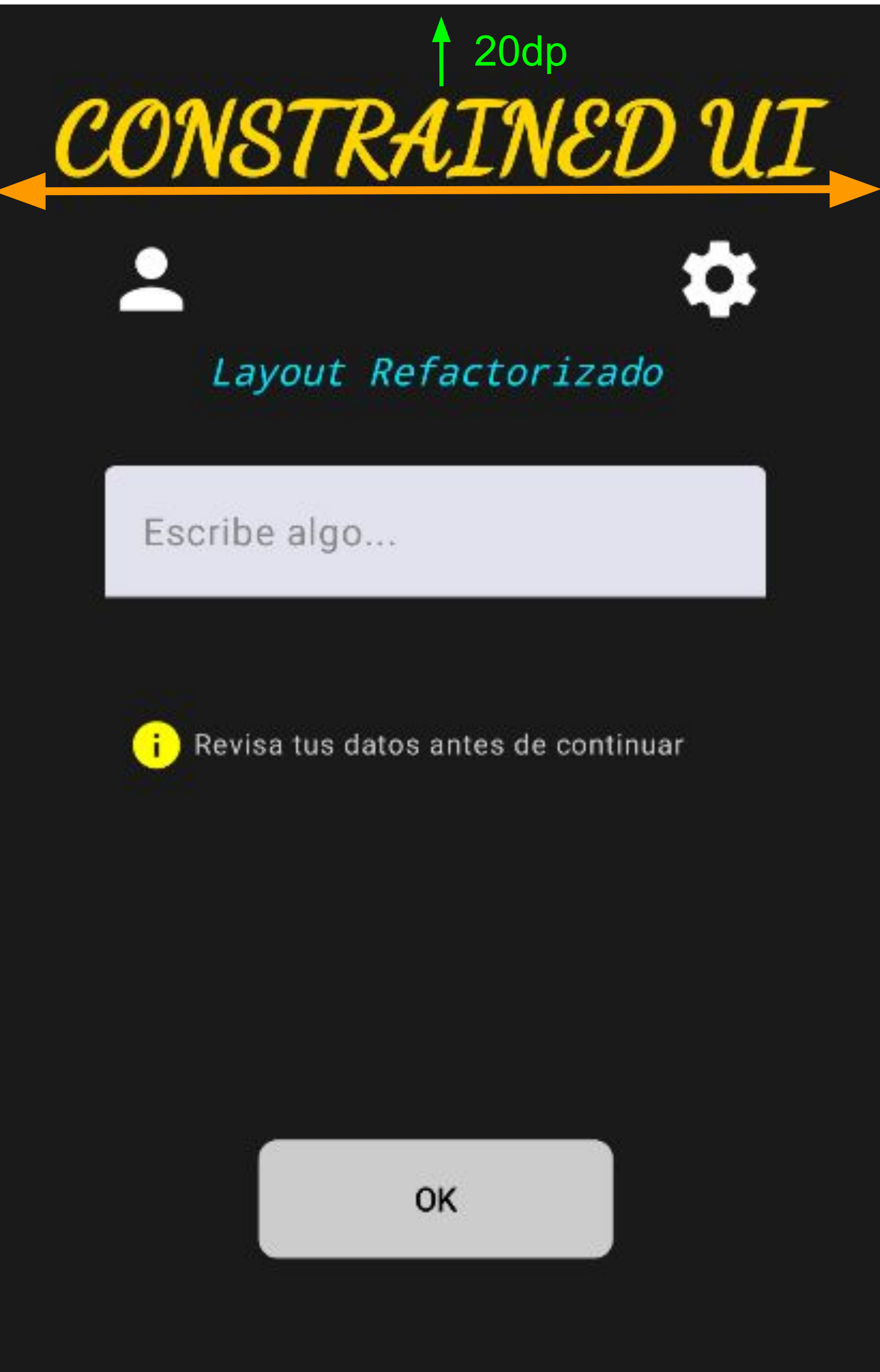
COMPOSE

```
MainActivity.kt build.gradle.kts (:app) × libs.versions.kts
You can use the Project Structure dialog to view and edit your project structure.

1  plugins {
2      alias(libs.plugins.android.application)
3      alias(libs.plugins.kotlin.compose)
4  }
5
6  android {
7      namespace = "com.example.pmdm_ej1"
8      compileSdk = 35
9
10     defaultConfig {
11         applicationId = "com.example.pmdm_ej1"
12         minSdk = 24
13         targetSdk = 35
14     }
15 }
```

```
implementation(libs.androidx.compose.ui.graphics)
implementation(libs.androidx.compose.ui.tooling.preview)
implementation(libs.androidx.compose.material3)
implementation(libs.androidx.constraintlayout.compose)
testImplementation(libs.junit)
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.androidx.espresso.core)
androidTestImplementation(platform(libs.androidx.compose.bom))
```

`implementation(libs.androidx.constraintlayout.compose)`



```
@Composable
fun MyScreen(modifier: Modifier = Modifier) {
    var textValue by remember { mutableStateOf( value = "") }

    Box(
        modifier = modifier
            .fillMaxSize()
            .background(Color(0xFF1A1A1A)), // Fondo casi negro para contraste
        contentAlignment = Alignment.Center
    ) {
        ConstraintLayout(
            modifier = Modifier
                .fillMaxSize()
                .padding( all = 16.dp)
        ) {
            // Referencias para los elementos del L.C.
            val (title, subtitle, iconPerson, iconSettings, textField, iconInfo, infoText, btn1) = createRefs()

            // Título
            Text(
                text = "CONSTRAINED UI",
                style = TextStyle(
                    color = Color(0xFFFFD600),
                    fontSize = 40.sp,
                    fontWeight = FontWeight.Black,
                    fontFamily = FontFamily.Cursive
                ),
                modifier = Modifier.constrainAs( ref = title) {
                    top.linkTo( anchor = parent.top, margin = 20.dp)
                    centerHorizontallyTo( other = parent)
                }
            )
        }
    }
}
```

Mantiene el texto cada vez que se recarga la UI

Contenedor para el layout

Crea el constraint Layout, que ocupe llenando al máximo dejando un espacio de 16 dp

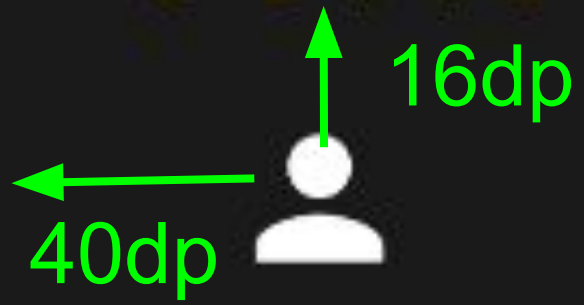
Define las referencias de cada elemento del CL para luego poder referenciarlos

1er elemento del CL con las siguientes constraints:

- vinculado por arriba a la parte de arriba del padre con un margen de 20 dp
- Centrado en horizontal respecto al padre



CONSTRAINED UI



Layout Refactorizado

Escribe algo...



Revisa tus datos antes de continuar

OK

```
// Icono de Persona
Icon(
    imageVector = Icons.Default.Person,
    contentDescription = null,
    tint = Color.White,
    modifier = Modifier
        .size(40.dp)
        .constrainAs(ref = iconPerson) {
            top.linkTo(anchor = title.bottom, margin = 16.dp)
            start.linkTo(anchor = parent.start, margin = 40.dp)
        }
)

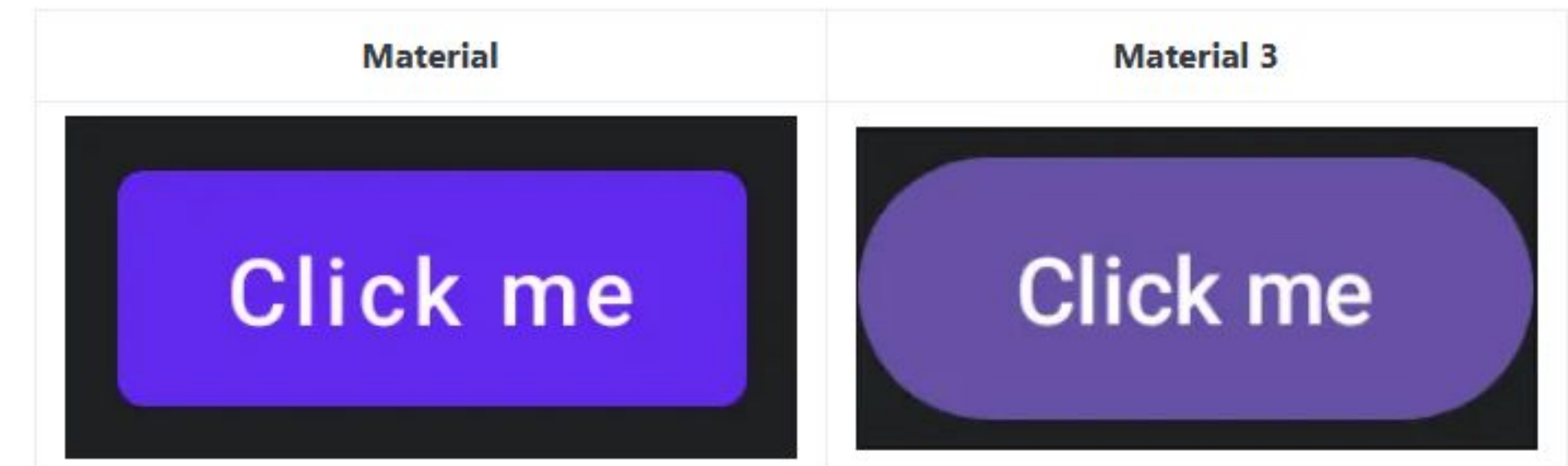
// Icono de Ajustes
Icon(
    imageVector = Icons.Default.Settings,
    contentDescription = null,
    tint = Color.White,
    modifier = Modifier
        .size(40.dp)
        .constrainAs(ref = iconSettings) {
            top.linkTo(anchor = title.bottom, margin = 16.dp)
            end.linkTo(anchor = parent.end, margin = 40.dp)
        }
)
```



BUTTON: Botones con su apariencia y una función asociada que se ejecuta al pulsarla

Button

```
@Composable
fun Button(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    shape: Shape = ButtonDefaults.shape,
    colors: ButtonColors = ButtonDefaults.buttonColors(),
    elevation: ButtonElevation? = ButtonDefaults.buttonElevation(),
    border: BorderStroke? = null,
    contentPadding: PaddingValues = ButtonDefaults.ContentPadding,
    interactionSource: MutableInteractionSource = remember { MutableInteractionSource() },
    content: @Composable RowScope.() -> Unit
)
```



```
@Composable
fun ButtonExample() {
    Button(onClick = { print("Hello") }) {
        Text(text = "Click me")
    }
}
```

Acción a realizar cuando se pulsa

JETPACK



COMPOSE

```
@Composable
fun LikeScreen(modifier: Modifier = Modifier) {
    // Estado para controlar si se ha dado al like
    var isLiked by remember { mutableStateOf( value = false) }
```

variable con el estado del botón, puesta a FALSE y que se recuerde.

```
Button(onClick = { isLiked = true }) {
    Icon(
        imageVector = Icons.Default.Favorite,
        contentDescription = null,
        modifier = Modifier.size(20.dp),
        tint = if (isLiked) Color.Red else Color.White
    )
    Spacer(Modifier.size(8.dp))
    Text("LIKE")
}
```

Cuando se haya pulsado, el corazón se pondrá rojo



```
if (isLiked) { Al pulsar aparece el texto
    Text(
        text = "has dado al like",
        fontSize = 18.sp,
        color = Color.Gray
    )
}
```

Buttons



Button

FilledTonalButton

OutlinedButton

ElevatedButton

TextButton

FloatingActionButton

SmallFloatingActionButton

LargeFloatingActionButton

ExtendedFloatingActionButton

RadioButton

IconButton



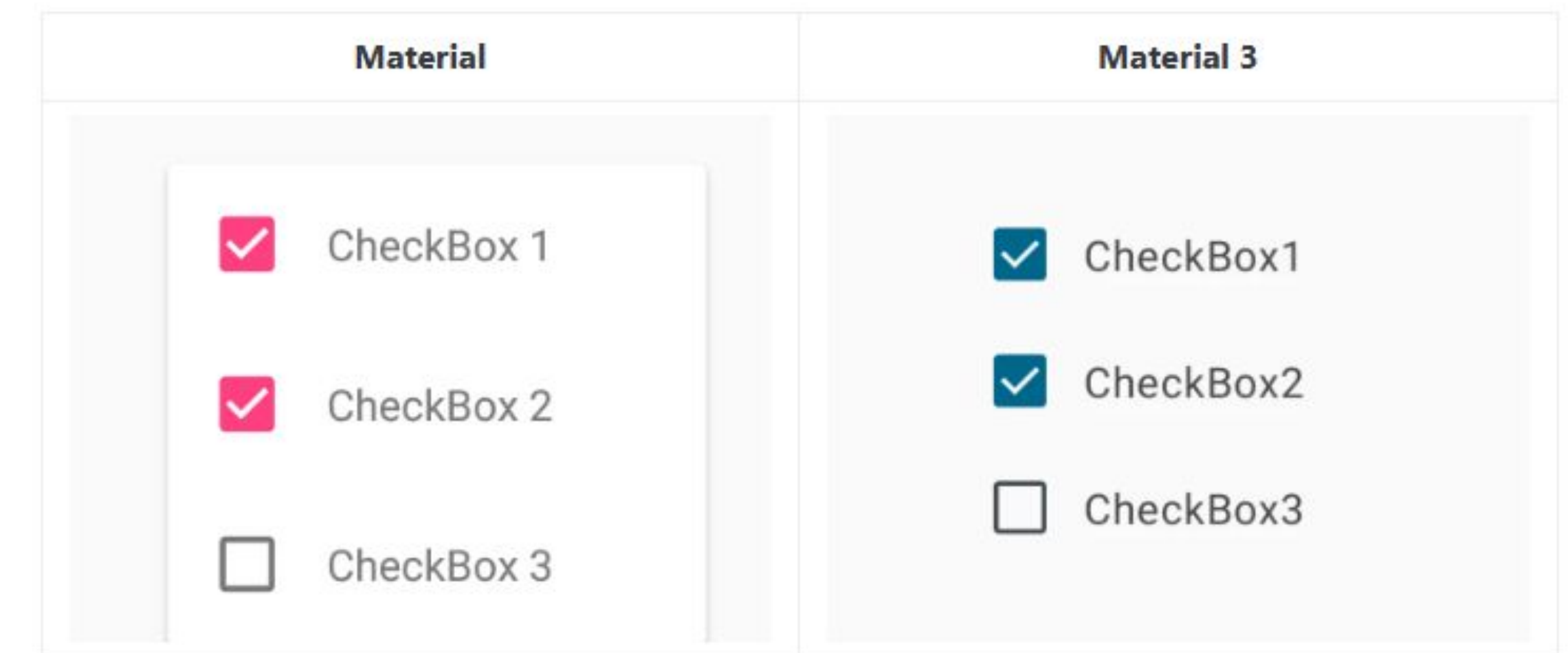
Ejercicio: Prueba usando diferentes tipos de botones las siguientes funcionalidades:

- Dos botones (lock y unlock) y que al pulsar cada uno de ellos se deshabilite a sí mismo y habilite el otro
- Un texto random que al pulsar un botón se ponga todo en mayúsculas y al volver a pulsarlo se ponga de nuevo en minúsculas (toggle)
- Un botón que tenga dos calaveras (a izquierda y derecha del texto) añada las calaveras con una imagen.
- Otras funcionalidades que se te ocurran.



CHECKBOX: Permiten seleccionar elementos, podemos leer su estado y actuar cuando se cambie.

Checkbox



```
@Composable
fun Checkbox(
    checked: Boolean,
    onCheckedChangeListener: ((Boolean) -> Unit)?,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    colors: CheckboxColors = CheckboxDefaults.colors(),
    interactionSource: MutableInteractionSource = remember { MutableInteractionSource() }
): Unit
```

```
import androidx.compose.material.Checkbox

@Composable
fun CheckboxDefault() {
    val checkedState = remember { mutableStateOf(true) }
    Checkbox(
        checked = checkedState.value,
        onCheckedChangeListener = { checkedState.value = it }
    )
}
```

marca el valor (true)

Al cambiar de valor actualiza su estado

JETPACK COMPOSE

```
// Estado para el color de fondo seleccionado (por defecto Gris)
var selectedColor by remember { mutableStateOf( value = Color.Gray) }
```

```
val colorOptions = listOf(
    "Gris" to Color.Gray,
    "Negro" to Color.Black,
    "Azul" to Color.Blue,
    "Verde" to Color.Green
)
```

Crea una lista inmutable con los colores disponibles para el fondo el `to` asigna los pares.

```
Column(
    horizontalAlignment = Alignment.Start
) {
```

```
    colorOptions.forEach { (name, color) ->
```

Bucle que recorre la lista con sus pares, en cada pasada tienen su valor.

```
        Row(
```

```
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(vertical = 2.dp)
        ) {
```

```
            Checkbox(
                checked = selectedColor == color,
                onCheckedChange = { if (it) selectedColor = color }
            )
```

```
            Text(
                text = name,
                fontSize = 14.sp,
                color = Color.Black
            )
        }
```

Se marca si `selectedColor` coincide con el del bucle (`color`)

al cambiar el estado, si está check (`it`) actualiza el `selectedColor` con el actual

```
                text = name,
                fontSize = 14.sp,
                color = Color.Black
            )
```

Después del checkbox pone el texto con el nombre del color

Elige el color de fondo:

☒ Gris

☐ Negro

☐ Azul

☐ Verde



SCAFFOLD: Plantilla base con estructura típica de MD para poner elementos en cada zona

```
@OptIn(...)markerClass = ExperimentalMaterial3Api::class)
@Composable
fun EjemploScaffold() {
    Scaffold(
        topBar = {
            TopAppBar(title = { Text(text = "Mi App") })
        },
        bottomBar = {
            BottomAppBar {
                Text(text = "Barra inferior")
            }
        },
        content = {
            innerPadding ->
            // Contenido de la pantalla
            Column(
                modifier = Modifier
                    .padding(paddingValues = innerPadding) // ajusta el contenido para
                    .fillMaxSize(),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(text = "Hola desde el contenido principal")
            }
        }
    )
}

@Preview(showBackground = true)
@Composable
fun EjemploScaffoldPreview() {
    PMDM_EJ1Theme {
        EjemploScaffold()
    }
}
```

The diagram illustrates the Scaffold structure with three distinct zones, each highlighted by a colored box:

- Top Bar (Red Box):** Contains the text "Mi App".
- Bottom Bar (Green Box):** Contains the text "Barra inferior".
- Main Content Area (Cyan Box):** Contains the text "Hola desde el contenido principal".



Ejercicio: realizar un activity similar al siguiente, para ello usar:

- Image y text que ya conocemos
- Scaffold para la estructura
- switch para los botones deslizantes
- Slider para el ajuste de brillo:

para visualizar el texto:

```
text = "Ajustar brillo de pantalla: ${(zoomLevel * 100)}.
```

para visualizar el slider

```
value = zoomLevel,  
onValueChange = { zoomLevel = it },
```





Ejercicio: Mezclar ROW y COLUMN para conseguir una interfaz de este estilo:

A mobile app login screen mockup. It has a light gray background. At the top, the text "Inicio de Sesión" is centered in a dark blue font. Below it are four circular icons: a cat face, an envelope, a person silhouette, and an information 'i'. These are followed by two light purple input fields with rounded corners. The first field is labeled "Usuario" and the second is labeled "Contraseña". Below these fields is a dark blue rounded rectangular button labeled "Confirmar". At the bottom, the text "¿Has olvidado tu contraseña?" is centered in a dark blue font.